In addition, RSpec has a generic matcher match, which can be applied to arrays, strings, or hashes. Typically the argument to match is a pattern and the expectation passes if the expected value fits the pattern. You can use other RSpec matchers to fill part of the pattern.

Some examples:

```
expect(string).to match(/regex/)
expect(array).to match([3, 5])
expect(hash).to match(color: a_string_starting_with("b"))
```

The last example shows that RSpec offers aliases for most of its matchers so they read more like natural language when used internally, so "a_string_starting_with" is an alias for "starts _with". This is where people's opinions about RSpec start to split—some people find this kind of linguistic wordplay elegant and clever, others find it confusing and overly complicated.

Which is a great lead in to RSpec's dynamic matcher syntax.

Often, an object has boolean methods that you want to test. In Ruby, the community standard is to end boolean methods with a ? as in: paperback?.

```
class Book
  def paperback?
    type == :paper
  end
end
```

You can test this method in RSpec using normal RSpec syntax, which would look something like this:

```
expect(book.paperback?).to be_truthy
```

And that's fine, it works. But if you read it out loud it sounds weird—more like how computers talk and not like how people talk. What you might want is to be able to write this:

```
expect(book).to be_a_paperback
```

Read that out loud, and it sounds like natural language.

So, in order to make that work in RSpec, you need to do… nothing. It already works.

When RSpec sees a matcher that starts with be, be_a, or be_an, it does some parsing on the name of the matcher and looks for a method in the object under test. If the matcher has no arguments, it looks for a predicate method that ends in a question mark, so expect(book).to be_a_paperback, looks for book.paperback? and failing that, book.paperbacks?. If the matcher has arguments, it looks for a regular method, so expect(book). to_be_published_at(Date.today), would look for book.published_at(Date.today). RSpec will do the same thing with have and has. The matcher expect(book).to have_cover will look for book.has_cover?, while expect(book).to have_author("Dave"), will look for book.has_author("Dave").

There are also a set of matchers that take a block, the general structure is expect { SOMETHING }.to MATCHER. The most common is probably expect { }.to raise_error(arg), where the argument is usually a Ruby exception class and the matcher passes if the block raises the expected error. The argument could also be a string or regular expression matching the error message.